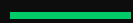


whISP

Internet Service Provider Management Platform



Implementation Guide

Architecture · Setup · API Reference · Deployment

VERSION

1.0

PLATFORM

Bangladesh

DATE

March 2026

STATUS

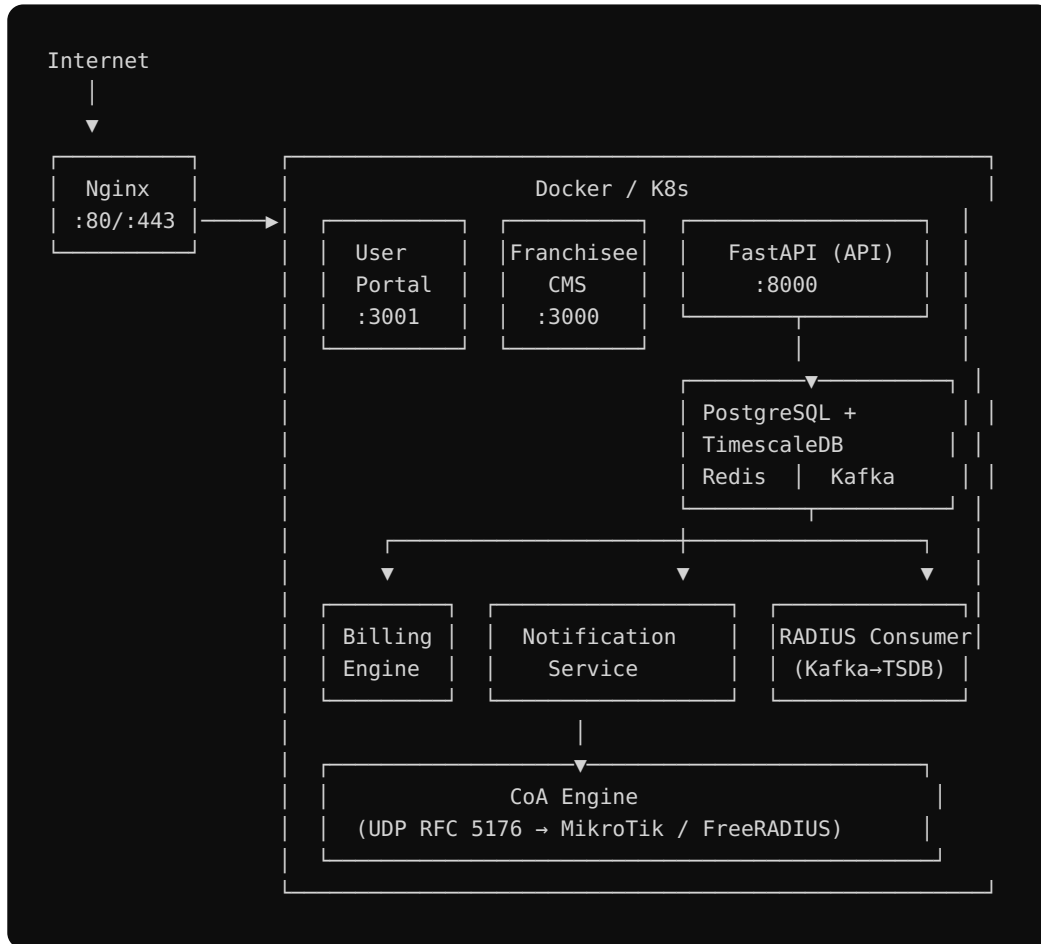
Internal Use

Table of Contents

- 1** Architecture Overview
- 2** Prerequisites
- 3** Local Development Setup
- 4** Environment Configuration
- 5** Database Setup & Migrations
- 6** Services Reference
 - 6.1 API Service · 6.2 Billing Service · 6.3 CoA Engine · 6.4 Notification Service · 6.5 RADIUS Consumer
- 7** API Endpoints Reference
 - Auth · Subscribers · Payments · Plans · Franchisees · Usage · OTT · Tickets · Admin
- 8** Payment Gateway Integration
 - 8.1 bKash PGW · 8.2 Nagad · 8.3 SSLCommerz · Post-payment flow
- 9** SMS Gateway (SSL Wireless)
- 10** RADIUS / FreeRADIUS Integration
- 11** OTT Partner Integration
- 12** Frontend Setup
 - 12.1 Franchisee CMS · 12.2 User Portal
- 13** Running Tests
- 14** Production Deployment (Kubernetes)
- 15** Troubleshooting

§1

Architecture Overview



Services

Service	Port	Role
api	8000	Main FastAPI REST backend
billing	—	APScheduler – expiry, auto-debit, reconciliation
coa-engine	—	RADIUS CoA/Disconnect UDP packets (RFC 5176)
notification	—	SMS (SSL Wireless) + FCM push via Kafka
radius-consumer	—	Kafka → TimescaleDB usage ingest
franchisee-cms	3000	React CMS for franchisee staff

<code>user-portal</code>	3001	React self-service portal for subscribers (Bengali)
--------------------------	------	---

Data Stores

Store	Use
PostgreSQL 15	All relational data (12 tables)
TimescaleDB extension	<code>usage_stats</code> hypertable (time-series bandwidth data)
Redis 7	JWT blacklist, session cache, rate-limit buckets
Kafka	Billing events, RADIUS accounting, notifications
FreeRADIUS (external)	PPPoE AAA; writes to a shared <code>radius</code> DB schema

Required Software

```
docker          >= 24.0
docker compose  >= 2.20
python          >= 3.11      # only for outside-Docker runs
node            >= 20        # only for frontend dev
git             >= 2.40
```

Python Tooling

```
pip install pipx
pipx install poetry
```

Bangladeshi Operator Registrations

Before going live you will need active accounts with the following providers:

Provider	Purpose	Sandbox URL
SSL Wireless	SMS OTP / alerts	sms.sslwireless.com
bKash PGW	Mobile payments	sandbox.payment.bka.sh
Nagad	Mobile payments	sandbox.nagad.com.bd
SSLCommerz	Card + MFS payments	sandbox.sslcommerz.com
Chorki	OTT partnership	Contact partner team
Hoichoi	OTT partnership	Contact partner team

Step 1 - Clone and configure

```
git clone <repo-url> whISP
cd whISP
cp .env.example .env
# Edit .env - see §4 for all variables
```

Step 2 - Start infrastructure services

```
docker compose up -d postgres redis zookeeper kafka
# Wait ~30 s until all four show "(healthy)"
docker compose ps
```

Step 3 - Run database migrations

```
docker compose run --rm api alembic upgrade head
```

Or locally (Python + deps installed):

```
cd services/api
pip install -r requirements.txt
PYTHONPATH=. alembic -c ../../alembic.ini upgrade head
```

Step 4 - Seed development data

```
docker compose run --rm api python scripts/seed_dev.py
# Creates: 3 franchisees, 5 plans, 100 subscribers
```

Step 5 - Start application services

```
docker compose up -d api billing coa-engine notification radius-consumer
docker compose logs -f api
# Expected: database_connected, redis_connected, startup_complete
```

Step 6 - Start frontend (optional)

```
# Franchisee CMS
cd frontend/franchisee-cms && npm install && npm run dev # http://
localhost:3000

# User portal (new terminal)
cd frontend/user-portal && npm install && npm run dev # http://
localhost:3001
```

Verify

```
curl http://localhost:8000/health
# {"status":"ok","redis":"connected","version":"..."}
```

Environment Configuration

Full annotated reference for `.env` (copy from `.env.example`):

Core

```
PLATFORM_NAME=whISP
PLATFORM_DOMAIN=yourdomain.com.bd
ENVIRONMENT=development      # development | staging | production
DEBUG=false
LOG_LEVEL=INFO
ADMIN_EMAIL=admin@yourdomain.com.bd
ADMIN_PASSWORD=<strong-password>
```

PostgreSQL

```
DATABASE_URL=postgresql+asyncpg://isp_user:isp_password@postgres:5432/
isp_platform
RADIUS_DB_URL=postgresql+asyncpg://radius_user:radius_pass@postgres:5432/
radius
DB_POOL_SIZE=20
DB_MAX_OVERFLOW=40
DB_POOL_TIMEOUT=30
DB_POOL_RECYCLE=1800
```

Note

The `postgresql+asyncpg://` prefix is required by the SQLAlchemy async engine. Plain `postgresql://` will fail.

Redis

```
REDIS_URL=redis://redis:6379/0
REDIS_MAX_CONNECTIONS=50
```

Kafka

```
KAFKA_BOOTSTRAP_SERVERS=kafka:9092
KAFKA_TOPIC_BILLING_EVENTS=billing-events
KAFKA_TOPIC_RADIUS_SYNC=radius-sync
```

```
KAFKA_TOPIC_NOTIFICATIONS=notifications
KAFKA_CONSUMER_GROUP=whisp-api
```

JWT

```
# Generate with: openssl rand -hex 32
JWT_SECRET_KEY=<64-char-hex-string>
JWT_ALGORITHM=HS256
JWT_EXPIRE_MINUTES=1440          # 24 h
JWT_REFRESH_EXPIRE_DAYS=30
```

Encryption

```
# Used to encrypt NID numbers and PPPoE passwords at rest
# Generate with: openssl rand -base64 32 | head -c 32
ENCRYPTION_KEY=<32-char-key>
```

bKash PGW

```
BKASH_BASE_URL=https://tokenized.sandbox.bka.sh/v1.2.0-beta
BKASH_APP_KEY=<from-bkash-dashboard>
BKASH_APP_SECRET=<from-bkash-dashboard>
BKASH_USERNAME=<sandbox-username>
BKASH_PASSWORD=<sandbox-password>
BKASH_CALLBACK_URL=https://yourdomain.com.bd/api/payments/bkash/callback
```

Nagad

```
NAGAD_BASE_URL=https://sandbox.nagad.com.bd
NAGAD_MERCHANT_ID=<merchant-id>
NAGAD_MERCHANT_PHONE=01XXXXXXXXX
NAGAD_PUBLIC_KEY=<base64-nagad-public-key>
NAGAD_PRIVATE_KEY=<base64-your-private-key>
NAGAD_CALLBACK_URL=https://yourdomain.com.bd/api/payments/nagad/callback
```

SSLCommerz

```
SSLCOMMERZ_BASE_URL=https://sandbox.sslcommerz.com
SSLCOMMERZ_STORE_ID=<store-id>
SSLCOMMERZ_STORE_PASSWD=<store-password>
SSLCOMMERZ_SUCCESS_URL=https://yourdomain.com.bd/api/payments/sslcommerz/
```

```
success
```

```
SSLCOMMERZ_FAIL_URL=https://yourdomain.com.bd/api/payments/sslcommerz/fail  
SSLCOMMERZ_CANCEL_URL=https://yourdomain.com.bd/api/payments/sslcommerz/  
cancel  
SSLCOMMERZ_IPN_URL=https://yourdomain.com.bd/api/payments/sslcommerz/ipn
```

SMS Gateway (SSL Wireless)

```
SMS_GATEWAY_URL=http://sms.sslwireless.com/pushapi/dynamic/server.php  
SMS_GATEWAY_API_KEY=<api-token>  
SMS_GATEWAY_SENDER_ID=whISP # max 11 chars, pre-approved  
SMS_GATEWAY_PROVIDER=ssl_wireless
```

FCM Push Notifications

```
FCM_SERVER_KEY=<firebase-server-key>
```

OTT Partners

```
CHORKI_BASE_URL=https://api.chorki.com  
CHORKI_API_KEY=<partner-api-key>  
CHORKI_PARTNER_ID=<partner-id>  
CHORKI_PARTNER_SECRET=<partner-secret>  
  
HOICHOI_BASE_URL=https://api.hoichoi.tv  
HOICHOI_API_KEY=<partner-api-key>  
HOICHOI_PARTNER_ID=<partner-id>  
HOICHOI_PARTNER_SECRET=<partner-secret>
```

RADIUS / CoA

```
RADIUS_COA_DEFAULT_PORT=3799  
RADIUS_COA_TIMEOUT=5.0
```

CORS

```
CORS_ORIGINS=["https://yourdomain.com.bd","https://cms.yourdomain.com.bd"]  
CORS_ALLOW_CREDENTIALS=true
```

Schema Overview

```

franchisees
├── franchisee_users      (staff login)
├── nas_devices           (MikroTik / FreeRADIUS gateways)
├── subscribers
│   ├── payment_transactions
│   ├── notifications
│   ├── ott_entitlements
│   ├── support_tickets
│   └── coa_commands      (audit trail of CoA packets)
plans                    (platform-wide or franchisee-scoped)
audit_logs               (immutable event journal)
usage_stats (TimescaleDB) (hypertable, indexed by time)
  
```

Running Migrations

```

alembic upgrade head      # apply all pending
alembic current           # show current revision
alembic history --verbose # full history
alembic downgrade -1      # rollback one step
  
```

Creating a New Migration

```

alembic revision --autogenerate -m "add_field_xyz"
# Edit the generated file in alembic/versions/ to verify
alembic upgrade head
  
```

TimescaleDB Hypertable

The `usage_stats` table must be converted to a hypertable after the first migration. Run once manually or in migration `0002` :

```

SELECT create_hypertable('usage_stats', 'time', if_not_exists => TRUE);
SELECT add_retention_policy('usage_stats', INTERVAL '90 days');
SELECT add_compression_policy('usage_stats', INTERVAL '7 days');
  
```

6.1 API Service (`services/api`)

Entry point: `app/main.py` · Start: `uvicorn app.main:app --host 0.0.0.0 --port 8000`

Startup sequence: connect PostgreSQL pool → connect Redis → log `startup_complete` .

Dependency	Purpose
fastapi + uvicorn	HTTP framework
sqlalchemy[asyncio] + asyncpg	Async ORM + DB driver
passlib[bcrypt]	Password hashing
python-jose[cryptography]	JWT tokens
httpx	Async HTTP calls to gateways
redis[asyncio]	Token blacklist + cache
aiokafka	Kafka producer for events
structlog	Structured JSON logging

6.2 Billing Service (`services/billing`)

Entry point: `app/__main__.py` · Start: `python -m app`

Job	Schedule	Action
check_expiry_warnings	Every hour	SMS at T-24h and T-2h before plan expiry
enforce_expiry	Every 5 min	Auto-suspend expired subscribers
process_auto_debit	Every 15 min	Execute bKash recurring debit
reconcile_revenue	Daily 02:00 BDT	Monthly revenue report per franchisee
check_agreement_health	Daily 03:00 BDT	Validate bKash agreement status

Timezone

All schedules are Asia/Dhaka (UTC+6).

6.3 CoA Engine (`services/coa-engine`)

Sends RFC 5176 Change-of-Authorization packets over UDP to MikroTik routers.

Packet Type	Code	Use
Disconnect-Request	40	Force PPPoE session termination
CoA-Request	43	Apply rate-limit (speed throttling)

MikroTik VSA (vendor ID 14988) is used for rate-limit:

```
Mikrotik-Rate-Limit = "10240k/5120k" .
```

Triggered by: subscriber suspension · plan expiry · quota exceeded · payment completion.

6.4 Notification Service (`services/notification`)

Runs two concurrent async loops:

- **DB drain loop** (every 5 s): polls `notifications` table for pending records, sends via SMS/push, updates status.
- **Kafka consumer loop:** consumes real-time events, inserts notification record, sends immediately.

Rate limiting: token-bucket algorithm, default 10 SMS/second.

6.5 RADIUS Consumer (`services/radius-consumer`)

Kafka topic → TimescaleDB usage ingest.

Event	Action
Acct-Status-Type = Start	Insert session, mark subscriber online
Acct-Status-Type = Interim-Update	Update quota_used_bytes, check quota limit
Acct-Status-Type = Stop	Mark session ended, write final usage to TimescaleDB

When `quota_used_bytes >= plan.quota_gb × 1 073 741 824` , a CoA rate-limit request is published.

API Endpoints Reference

All endpoints are prefixed `/api/` behind Nginx. Direct base URL: `http://localhost:8000`.

Include token in every request: `Authorization: Bearer <access_token>`

Authentication

Method	Path	Description
POST	/auth/login	Returns access + refresh tokens
POST	/auth/refresh	Exchange refresh → new access token
POST	/auth/logout	Blacklist current token in Redis
POST	/auth/register/subscriber	Subscriber self-registration
GET	/auth/me	Current user info

```
curl -X POST http://localhost:8000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@example.com","password":"pass","actor_type":"admin"}'

# Response: {"access_token":"eyJ...", "refresh_token":"eyJ...", "expires_in":
86400}
```

Subscribers

Method	Path	Description
GET	/subscribers	List (role-scoped to franchisee)
POST	/subscribers	Create + RADIUS sync
GET	/subscribers/{id}	Detail + active session info
PATCH	/subscribers/{id}	Update profile / plan
DELETE	/subscribers/{id}	Soft delete + RADIUS cleanup
POST	/subscribers/{id}/suspend	Suspend + CoA disconnect
POST	/subscribers/{id}/reactivate	Reactivate + RADIUS resync

POST	/subscribers/{id}/reset-password	Regenerate PPPoE + portal passwords
GET	/subscribers/{id}/sessions	Active PPPoE sessions from radacct

Payments

Method	Path	Description
GET	/payments	List transactions
GET	/payments/{id}	Transaction detail
POST	/payments/bkash/initiate	Returns bKash redirect URL
POST	/payments/bkash/callback	IPN - executes payment + subscription
POST	/payments/bkash/agreement/create	Initiate bKash auto-debit agreement
POST	/payments/bkash/agreement/callback	Store agreementID
POST	/payments/nagad/initiate	Initiate Nagad payment
POST	/payments/nagad/callback	Nagad IPN callback
POST	/payments/sslcommerz/initiate	SSLCommerz checkout
POST	/payments/sslcommerz/success	Success redirect handler
POST	/payments/sslcommerz/ipn	SSLCommerz IPN (server-to-server)
POST	/payments/manual	Record manual payment (admin/franchisee)

Plans

Method	Path	Description
GET	/plans	List (public + franchisee-scoped)

POST	/plans	Create + sync to FreeRADIUS radgroupreply
GET	/plans/{id}	Plan detail
PATCH	/plans/{id}	Update + re-sync if speeds changed
DELETE	/plans/{id}	Deactivate (blocked if active subscribers)

Franchisees (admin only)

Method	Path	Description
GET / POST	/franchisees	List / Create
GET / PATCH	/franchisees/{id}	Detail / Update
POST	/franchisees/{id}/approve	Approve pending franchisee
POST	/franchisees/{id}/suspend	Suspend franchisee
POST	/franchisees/{id}/rotate-secret	Rotate RADIUS secret + update all NAS
POST	/franchisees/{id}/credit	Add balance credit
GET	/franchisees/{id}/mikrotik-config	Generate RouterOS script (.rsc)

Usage Analytics

Method	Path	Description
GET	/usage/franchisee/overview	Active sessions, bandwidth, revenue
GET	/usage/franchisee/chart?period=24h	Time-series (6h 24h 72h 7d)
GET	/usage/franchisee/top-consumers	Top 10 by bytes this month
GET	/usage/franchisee/sessions	Live PPPoE session list
GET	/usage/subscriber/{id}	Per-subscriber quota % + 7-day history

OTT / Support / Admin

Method	Path	Description
GET	/ott/entitlements	List subscriber OTT entitlements
GET	/ott/launch/{partner}	Generate SSO URL (Chorki / Hoichoi)
POST	/ott/provision	Manually provision entitlement
POST	/ott/revoke/{id}	Revoke entitlement
GET / POST	/tickets	List / Create support tickets
PATCH	/tickets/{id}	Update status / priority
POST	/tickets/{id}/reply	Add message to thread
GET	/admin/stats	Platform-wide KPIs
GET	/admin/audit	Audit log
POST	/admin/broadcast	Mass SMS/push notification
GET	/health	Service health check
GET	/metrics	Prometheus text-format metrics

8.1 bKash PGW

Flow:

1. `POST /payments/bkash/initiate` → API calls bKash `/tokenized/checkout/create` → returns `bkashURL` (redirect user here)
2. User completes payment on bKash app
3. bKash calls `POST /payments/bkash/callback` (IPN) → API calls `/tokenized/checkout/execute` → if `transactionStatus = "Completed"` : run `_complete_subscription()`

Sandbox test credentials: customer wallet `01877722345` , PIN `12121` .

Auto-debit: initiate agreement via `/payments/bkash/agreement/create` , store returned `agreementID` in `subscribers.bkash_agreement_id` . Billing engine charges automatically on renewal.

8.2 Nagad

Nagad uses RSA encryption. Generate a key pair:

```
openssl genrsa -out nagad_private.pem 2048
openssl rsa -in nagad_private.pem -pubout -out nagad_public.pem
# Base64-encode both for .env
base64 -w 0 nagad_private.pem > nagad_private.b64
```

Share your public key with Nagad. Put their public key in `NAGAD_PUBLIC_KEY` and your private key in `NAGAD_PRIVATE_KEY` (both base64-encoded).

8.3 SSLCommerz

SSLCommerz handles Visa/Mastercard, bKash, Nagad, and Rocket in a single checkout page.

Critical - IPN Validation

SSLCommerz sends a POST to `SSLCOMMERZ_IPN_URL` . Always verify the `val_id` against the SSLCommerz validation endpoint before crediting the subscriber. Never skip this in production.

Post-payment flow - `_complete_subscription()`

Called by all gateway callbacks on success:

```
1. subscriber.plan_id = plan_id
2. subscriber.quota_used_bytes = 0
3. subscriber.plan_expires_at = now + timedelta(days=plan.validity_days)
4. subscriber.status = "active"
5. await radius_sync(subscriber)      # UPDATE radcheck + radusergroup
6. await coa_reconnect(subscriber)    # CoA-Request or
   Disconnect+reconnect
7. await provision_ott(subscriber)    # Chorki / Hoichoi if plan includes
   them
8. await queue_sms(subscriber, "activation")
```

SMS Gateway (SSL Wireless)

SSL Wireless covers all four Bangladesh mobile operators: Grameenphone, Robi/Airtel, Banglalink, Teletalk.

API Parameters

Field	Value
api_token	Your API token from SSL Wireless dashboard
sid	Sender ID (pre-approved, max 11 chars)
msisdn	Recipient in +8801XXXXXXXX format
sms	Message body (Unicode / Bengali supported)
csmsid	Your unique message reference ID

Phone Number Normalisation

Input	Normalised
01712345678	+8801712345678
1712345678	+8801712345678
8801712345678	+8801712345678
+8801712345678	+8801712345678

Test the Gateway Directly

```
curl -X POST "http://sms.sslwireless.com/pushapi/dynamic/server.php" \
  -d "api_token=$SMS_GATEWAY_API_KEY" \
  -d "sid=$SMS_GATEWAY_SENDER_ID" \
  -d "msisdn="+8801712345678" \
  -d "sms=Test message" \
  -d "csmsid=test001"
```

Recommended Templates

Event	Template (Bengali)
-------	--------------------

Activation	আপনার whISP সংযোগ সক্রিয়। প্যাকেজ: {plan}, মেয়াদ: {expiry}
Expiry T-24h	আপনার ইন্টারনেট {expiry} তারিখে শেষ হবে। নবায়ন করুন: {url}
Payment OK	পেমেন্ট গ্রহণ হয়েছে ₳{amount}। ধন্যবাদ।
Suspension	আপনার সংযোগ সাময়িকভাবে স্থগিত করা হয়েছে।

Character Limits

Standard SMS = 160 chars. Bengali Unicode = 70 chars per segment. Keep automated messages under 70 chars to avoid multi-part charges.

Sender ID Approval

Contact SSL Wireless to pre-approve your sender ID. Masking (showing a name instead of a number) requires BTRC registration. Allow 3–5 business days.

Shared Database Schema

Table	Purpose
radcheck	PPPoE credentials (Cleartext-Password)
radusergroup	Maps username → plan group
radgroupreply	Speed limits per group (set when creating a Plan)
radacct	Live + historical accounting sessions (written by FreeRADIUS)

RADIUS Sync - SQL

```
-- 1. Upsert credentials
INSERT INTO radcheck (username, attribute, op, value)
VALUES ($1, 'Cleartext-Password', ':=', $2)
ON CONFLICT (username, attribute) DO UPDATE SET value = EXCLUDED.value;

-- 2. Set group (plan)
DELETE FROM radusergroup WHERE username = $1;
INSERT INTO radusergroup (username, groupname, priority) VALUES ($1, $2,
1);
```

FreeRADIUS Configuration

```
# sql.conf
server    = "postgres-host"
port      = 5432
login     = "radius_user"
password  = "radius_pass"
radius_db = "radius"
```

```
# sites-enabled/default
authorize { sql }
authenticate { Auth-Type PAP { pap } }
accounting { sql }
```

MikroTik - Enable CoA

```
# RouterOS terminal
/radius incoming set accept=yes port=3799
/radius add address=<api-server-ip> secret=<nas_devices.secret> service=ppp
```

Secret Match

The `secret` field in the `nas_devices` database table must exactly match the RADIUS secret configured on the MikroTik router. A mismatch causes all CoA packets to be silently rejected.

Chorki - Provisioning

```
POST {CHORKI_BASE_URL}/partner/provision
Authorization: Bearer {CHORKI_API_KEY}
{
  "partner_id": "{CHORKI_PARTNER_ID}",
  "subscriber_id": "<whisp-subscriber-uuid>",
  "plan_id": "<chorki-plan-id>",
  "phone": "+8801XXXXXXXXX"
}
# Response stores partner_user_id + partner_subscription_id
```

Chorki - SSO Launch

```
GET {CHORKI_BASE_URL}/partner/sso-token?subscriber_id=...
# Returns: { "launch_url": "https://chorki.com/watch?token=..." }
```

Hoichoi - Provisioning

```
POST {HOICHOI_BASE_URL}/api/partner/subscribe
{
  "partner_id": "{HOICHOI_PARTNER_ID}",
  "msisdn": "+8801XXXXXXXXX",
  "package": "monthly"
}
```

Graceful Degradation

If OTT API keys are not configured, the platform returns a placeholder URL and logs a warning — it does not fail the payment flow. This allows testing without live OTT credentials.

12.1 Franchisee CMS (`frontend/franchisee-cms`) — Port 3000

Dark-theme management dashboard for franchisee staff. Sidebar navigation, all pages fully implemented.

```
cd frontend/franchisee-cms
npm install
npm run dev      # http://localhost:3000
```

Page	Route	Key Features
Login	/login	Email + password → JWT → dashboard
Dashboard	/dashboard	Stat cards, 24h bar chart (Recharts), PPPoE sessions, top consumers. Auto-refreshes every 30 s
Subscribers	/subscribers	Searchable table, quota progress bars, Add/Suspend/Reactivate/Reset PWD
Plans	/plans	Card grid, Create/Edit modal (speeds, throttle, cap, OTT checkboxes), Deactivate
Usage	/usage	Area chart (6h/24h/72h/7d), top-10 ranked list, live sessions table
Billing	/billing	Revenue summary cards, date-range filter, transactions table with gateway colour badges
Devices	/devices	NAS table, Register modal, Test CoA (shows latency), Download RouterOS config (.rsc)
Tickets	/tickets	Ticket list with priority/status badges, threaded conversation modal, reply form

Both hooks are pre-built:

- `useAuth()` – JWT token storage, login/logout, auto-refresh
- `useApi()` – Authenticated API calls with error handling and fallback

12.2 User Portal (`frontend/user-portal`) — Port 3001

Mobile-first Bengali-language self-service portal. Bottom tab navigation.

```
cd frontend/user-portal
npm install
npm run dev      # http://localhost:3001
```

Page	Route	Key Features
Login	/login	PPPoE credentials form
Home	/home	Animated SVG quota ring (colour: green → orange → red), remaining GB in Bengali digits, expiry countdown warning ≤ 3 days
Payment	/payment	Plan selector (highlights current), bKash/Nagad/Card method picker, gateway redirect, payment history table
OTT	/ott	2×2 card grid (Chorki/Hoichoi/Binge/Toffee), active → Watch Now, inactive → Upgrade
Support	/support	4-category quick-tap grid, ticket form, ticket list, chat-bubble conversation thread with reply input

Vite Proxy (dev only)

Both portals proxy `/api` to the backend automatically in development:

```
// vite.config.js
proxy: { '/api': { target: 'http://localhost:8000', changeOrigin: true } }
```

In production, Nginx handles this routing (see §14).

Install Test Dependencies

```
cd services/api
pip install -r requirements.txt -r requirements-test.txt
```

Run Tests

```
pytest -v                                # all tests
pytest tests/test_auth.py -v            # specific file
pytest tests/test_sms.py -v
pytest --cov=app --cov-report=term-missing # with coverage
```

Test Architecture

- Uses **SQLite in-memory** (aiosqlite) — no live PostgreSQL required
- Each test gets a **transactional session** rolled back on teardown (fast, isolated)
- SMS and HTTP calls are **mocked** with `unittest.mock` — no real gateway calls
- `conftest.py` overrides the `get_db` FastAPI dependency automatically

Adding Tests for New Routers

```
import pytest
from httpx import AsyncClient

@pytest.mark.asyncio
async def test_my_endpoint(client: AsyncClient):
    resp = await client.get(
        "/my-endpoint",
        headers={"Authorization": "Bearer <test-token>"}
    )
    assert resp.status_code == 200
```

Production Deployment (Kubernetes)

Prerequisites

```
kubectl version    # >= 1.28
helm version       # >= 3.12
# Running PostgreSQL 15 + TimescaleDB (RDS or self-managed)
# Running Redis cluster
# Running Kafka cluster (MSK or self-managed)
```

Step 1 - Namespace and Secrets

```
kubectl create namespace whisp
kubectl create secret generic whisp-env \
  --namespace whisp \
  --from-env-file=.env
```

Step 2 - Run Migrations

```
kubectl apply -f - <<EOF
apiVersion: batch/v1
kind: Job
metadata:
  name: whisp-migrate
  namespace: whisp
spec:
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: migrate
          image: <registry>/whisp-api:latest
          command: ["alembic", "upgrade", "head"]
          envFrom:
            - secretRef:
                name: whisp-env
EOF
kubectl wait --for=condition=complete job/whisp-migrate -n whisp --
timeout=120s
```

Step 3 - Apply Manifests

```
kubectl apply -f infrastructure/kubernetes/deployment.yaml
```

This creates Deployments + Services for all 5 services, and an HPA for the API (2 → 10 replicas at 70% CPU).

Step 4 - Ingress

```
kubectl apply -f - <<EOF
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: whisp-ingress
  namespace: whisp
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
    nginx.ingress.kubernetes.io/proxy-body-size: "10m"
spec:
  tls:
    - hosts: [yourdomain.com.bd]
      secretName: whisp-tls
  rules:
    - host: yourdomain.com.bd
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service: { name: whisp-api, port: { number: 8000 } }
          - path: /cms
            pathType: Prefix
            backend:
              service: { name: whisp-franchisee-cms, port: { number: 3000 } }
          - path: /
            pathType: Prefix
            backend:
              service: { name: whisp-user-portal, port: { number: 3001 } }
EOF
```

Step 5 - Verify

```
kubectl get pods -n whisp
kubectl logs -n whisp deployment/whisp-api --tail=50
curl https://yourdomain.com.bd/api/health
```

Build & Push Images

```
docker build -t <registry>/whisp-api:latest services/api/ && docker push  
<registry>/whisp-api:latest  
docker build -t <registry>/whisp-billing:latest services/billing/ && docker  
push <registry>/whisp-billing:latest  
# Repeat for coa-engine, notification, radius-consumer
```

database_connection_failed on startup

```
docker compose exec api python -c \  
    "import asyncpg, asyncio; asyncio.run(asyncpg.connect('$DATABASE_URL'))"
```

Common causes: wrong hostname (`postgres` inside Docker, `localhost` outside) · missing `postgresql+asyncpg://` prefix · TimescaleDB extension not installed.

redis_unavailable_cannot_blacklist_token

```
docker compose exec redis redis-cli ping    # should return PONG
```

Non-fatal in dev but **must be resolved in production** — tokens remain valid after logout without Redis.

SMS not delivered

1. Check `notifications` table: `SELECT * FROM notifications WHERE status='failed' ORDER BY created_at DESC LIMIT 10;`
2. Check the `error` column for the raw gateway response
3. Verify `SMS_GATEWAY_API_KEY` and `SMS_GATEWAY_SENDER_ID` are set
4. Confirm sender ID is approved by SSL Wireless for your account
5. Test the API directly (see §9)

CoA disconnect not working

1. Verify MikroTik: `/radius incoming print` — confirm `accept=yes`
2. Check UDP port 3799 is open: `nc -u <nas-ip> 3799`
3. Check `coa_commands` table for `success=false` + `error` column
4. Confirm `nas_devices.secret` matches the RouterOS RADIUS secret exactly

bKash callback not receiving

bKash requires `BKASH_CALLBACK_URL` to be publicly reachable over HTTPS. For local dev use a tunnel:

```
ngrok http 8000
# Set BKASH_CALLBACK_URL=https://<ngrok-subdomain>.ngrok.io/payments/bkash/
callback
```

alembic.ini not found

Always run Alembic from the repo root, or pass `-c` :

```
alembic -c /home/user/whISP/alembic.ini upgrade head
```

Subscriber shows online but has no internet

RADIUS sync succeeded but CoA was not sent or was ignored. Check `coa_commands` table. Manually trigger:

```
docker compose exec coa-engine python -c "
from app.coa import send_disconnect
import asyncio
asyncio.run(send_disconnect('<nas-ip>', '<secret>', '<username>'))
"
```

Kafka consumer lag growing

```
docker compose exec kafka kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --group whisp-api \
  --describe
```

If LAG is growing check `radius-consumer` and `notification` service logs for errors.